Lecture 3

Learning resources

🎸 How do I learn to program?

What should I be learning and how?

How do other people go about it?

What do they do?

How do they do it?

What works and what does not?

Programming = Problem Solving

let's rephrase the question

How do I learn to solve problems?

What is Programming?

Procedural Task: learn the sequence of activities that allow you to achieve a goal.

Conceptual Task: understand the principles that govern the domain and interrelations between pieces of knowledge

Find the two largest elements of a list of positive integers

A Procedural Solution

1. Set `max1`, `max2` to `0.`
2. If there are no remaining elements, stop
3. Take the next element call it `value`.
4. Find the smallest of `max1` and `max2` and if it is smaller than `value` set it to `value`.
5. Go to `step 2.`

Ouch: step 4 is another procedure that needs to be made explicit

A procedural solution in Python

```
max1 = max2 = 0
values = [1, 3, 1000, 2]

for value in values:
    if max1 < value < max2:
        max1 = value
        continue
    if max2 < value < max1:
        max2 = value
        continue
    if max1 < value and max1 <= max2:
        max1 = value
        continue
    if max2 < value and max2 <= max1:
        max2 = value
        continue

print(max1, max2)
```

A Conceptual Solution

1. Sort the list in descending order
2. Take the first two elements

Yep, that's all!

```
values = [1, 3, 1000, 2]
values = sorted(values, reverse=True)
max1, max2 = values[:2]
print(max1, max2)
```

In what way are these different?

Both snippets print the same

```
values = [1, 3, 1000, 2]
values = sorted(values, reverse=True)
max1, max2 = values[:2]
print(max1, max2)
```

But when is it better to use this:

```
values = [1, 3, 1000, 2]
values.sort(reverse=True)
max1, max2 = values[:2]
print(max1, max2)
```

Understanding this **is** programming!

Heresy Warning!

You don't need "algorithms"!

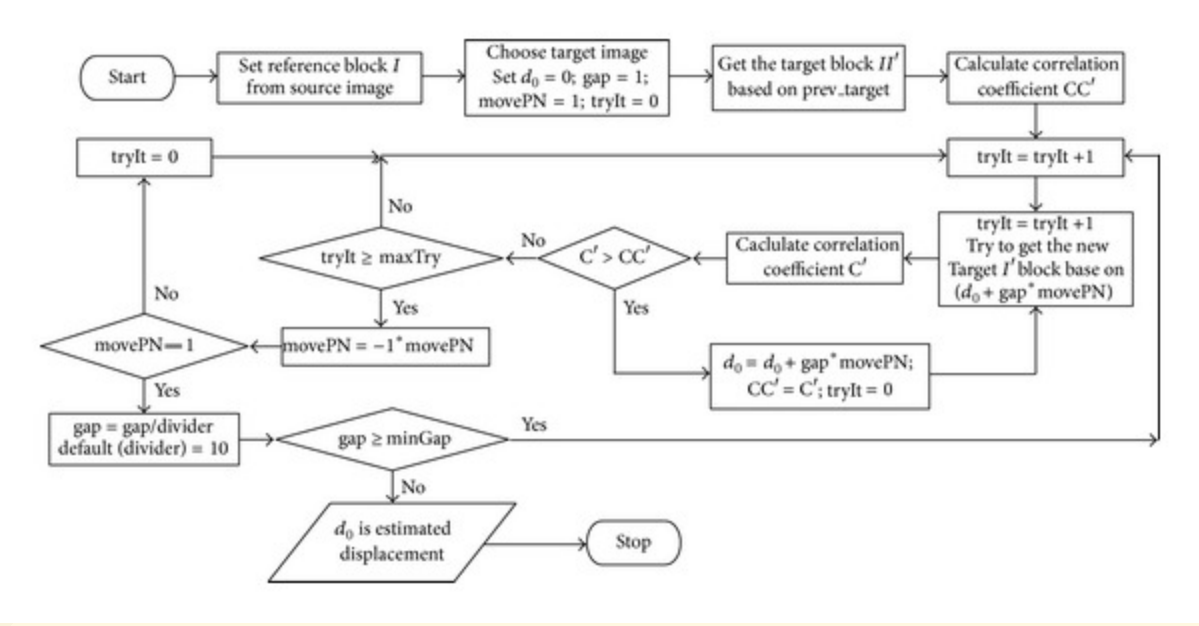
We don't need no Algorithmics!

We don't need no Thought Control!

Hey Algos! Leave me Alone!

Special note on FlowCharts

FLOWCHARTS are an inefficient way to demonstrate a thought process. Stay away from flowcharts!



Disclaimer

1. Some people - perhaps `1/100` of those that need programming will have to write procedural, algorithmic code.
2. They are **TRUE** heroes; they create the concepts that you and I need to use!
3. They are **TRUE** heroes, they make you and I more productive than we ever imagined.
4. But **YOU** don't have to do that.

Start thinking in terms of data "structures"

Which data structure

solves my problem right away?

Every solution means applying the right concepts and data representations to the problem.

In the majority of cases the representations exist - you have to recognize and apply them correctly!

So how do I learn problem-solving?

1. Should I read more books?
2. Should I take more classes?

Evaluate books

Get a book that is not too introductory. Bad choices:

- *Python for the CompleteIdiot*,
- *Learn to program in 24 hours*
- *Python for this or that* aka *Python for Bioinformatics*

Learn to program first, then learn to apply it to a topic.

Stay away from books with lenghty and "full" coding examples!

80/20 rule

Each program is 80% routine stuff with only 20% interesting and "actual" work.

Books that have extended code examples are full of useless and distracting content.

Plus they force you to think their way.

You can't reuse that code without retyping (copying) it.

Many books are unexpectedly bad

Use improper language, bad habits, sloppy definitions

See the my rant in the video 😈

Get a book that is readable

You can access immense amounts of information on the web that previously used to be in book.

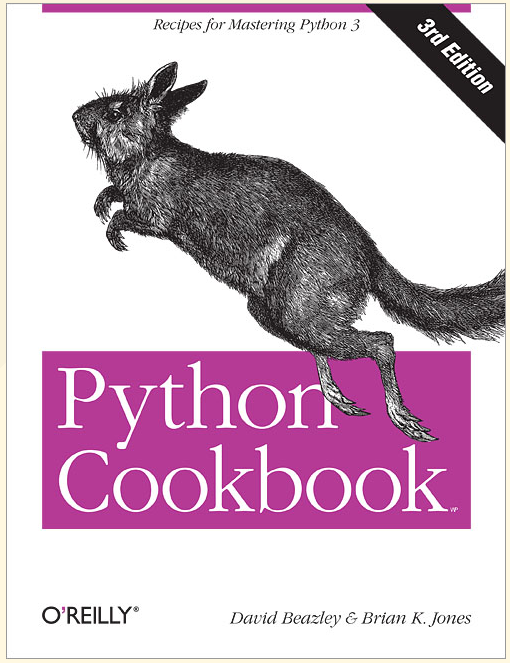
1. Definitions
2. Help
3. Examples

Your book should not need to reproduce that. Your book should be on what you can't read on the web in one place.

The official Python website has excellent documentation and tutorials!

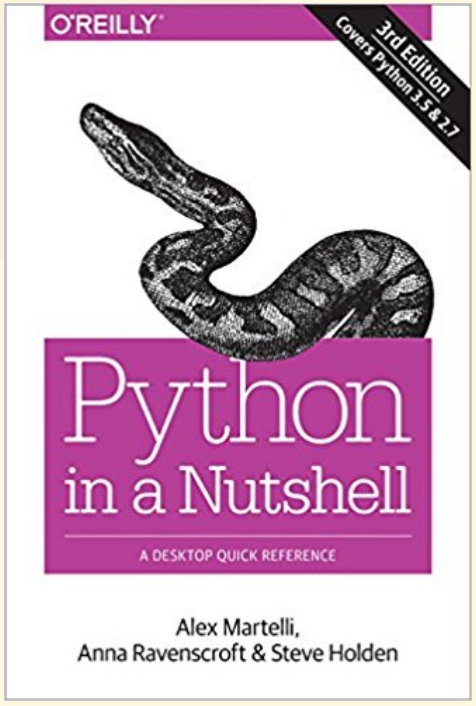
Book: Python Cookbook

1. **Python Cookbook** - Fun and engaging examples of how to use Python in various, realistic and exciting circumstances. Highly recommended!



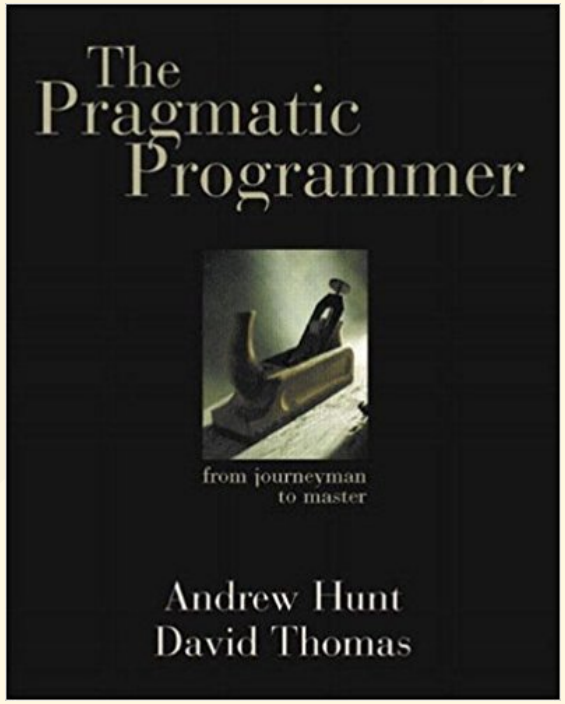
Book: Python in a Nutshell

1. **Python, in a Nutshell** - for advanced beginners - when you feel the urge to understand more deeply what happens under the hood. Accurate language, proper definitions.



Book: Pragmatic Programmer

1. **Pragmatic Programmer -** timeless advice on how to write better code.



Online Resources

Over the years I found myself regularly reading the following:

- Planet Python: http://planetpython.org/
- Pycoders Weekly: http://pycoders.com/
- Python Module of Week: https://pymotw.com/3/
- Python Tutorial: https://docs.python.org/3/tutorial/